Name- Aditya

Roll no.- 2401010134

Course- B.tech CSE Core (Section C)

Subject- Analysis and design of Algorithms Lab

Assignment 4

Workspace Bootstrap & Environment Setup

```
import time import matplotlib.pyplot

as plt from itertools import

permutations
```

Task 1: Backtracking Algorithm (Crew Scheduling)

```
# Flights (Flight ID, Start Time, End

Time) flights = [

    ('F1', 9, 11),

    ('F2', 10, 12),

    ('F3', 12, 14),

    ('F4', 13, 15)

]

crew_members = ['C1', 'C2', 'C3'] schedule = {} #

Check constraints def is_valid(crew, flight):     for

f, c in schedule.items():        if c == crew:

if not (flight[2] <= f[1] or flight[1] >= f[2]):

            return False

return True


# Backtracking function  def

backtrack(index):     if index
```

```
== len(flights):        return
True     flight = flights[index]
for crew in
crew_members:
        if is_valid(crew, flight):
schedule[flight] = crew


            if backtrack(index + 1):
return True            del
schedule[flight]     return False
backtrack(0) print("Backtracking
Schedule:") for f, c in
schedule.items():
    print(f[0], "->", c)
```

Output:

```
•••  Backtracking Schedule:
     F1 -> C1
     F2 -> C2
     F3 -> C1
     F4 -> C2
```

Task 2: Branch and Bound

```
schedule_bb = {} def branch_and_bound(index):
    if index == len(flights):
        return True

    flight = flights[index]     for
crew in crew_members:
```

```
        valid = True

        for f, c in schedule_bb.items():            if c ==
crew:                if not (flight[2] <= f[1] or flight[1]
>= f[2]):
                    valid = False                    break
if valid:
            schedule_bb[flight] = crew
if branch_and_bound(index + 1):
                return True
del schedule_bb[flight]     return
False  branch_and_bound(0)


print("\nBranch and Bound Schedule:") for f,
c in schedule_bb.items():
    print(f[0], "->", c)
```

Output:

```
Branch and Bound Schedule:
F1 -> C1
F2 -> C2
F3 -> C1
F4 -> C2
```

Task 3: String Matching (Naive)

```
def naive_search(text, pattern):
```

```
    n = len(text)     m = len(pattern)
count =
0     for i in range(nm+1):
match = True        for j in
range(m):
count  +=
1
            if   text[i+j]   !=   pattern[j]:
match = False              break        if
match:
          print("Pattern found at index", i)


    print("Total Comparisons:", count) text
= "crew scheduling system" pattern
= "crew"


print("\nNaive String Matching:")
naive_search(text, pattern)
```

Output:

```
Naive String Matching:
Pattern found at index 0
Total Comparisons: 23
```

Task 4: KMP Algorithm

```
def compute_lps(pattern):
lps = [0] * len(pattern)     length
= 0
```

```
    i = 1


    while i < len(pattern):        if pattern[i]
== pattern[length]:
            length += 1
lps[i] = length            i
+= 1        else:
if length != 0:
                length = lps[length - 1]
else:                lps[i] = 0                i
+= 1


    return lps def
kmp_search(text, pattern):     lps
= compute_lps(pattern)
i            = 0      j             = 0


    while i < len(text):


        if pattern[j] == text[i]:  i
            += 1            j += 1
            if j == len(pattern):
print("Pattern found at index", i-j)  j = lps[j-1]


        elif i < len(text) and pattern[j] != text[i]:
            if j != 0:
j            =
lps[j-1]             else:
i += 1
```

```
print("\nKMP String Matching:")

kmp_search(text, pattern)
```

Output:

```
KMP String Matching:
Pattern found at index 0
```

Task 5: Performance Comparison

```
text_large = "crew scheduling system " * 1000

pattern = "crew" start = time.time()

naive_search(text_large, pattern) end =

time.time()


print("Naive Time:", end - start)


start = time.time() kmp_search(text_large,

pattern) end

= time.time()


print("KMP Time:", end - start)
```

Output:

```
Pattern found at index 0
Pattern found at index 23
Pattern found at index 46
Pattern found at index 69
Pattern found at index 92
Pattern found at index 115
Pattern found at index 138
Pattern found at index 161
Pattern found at index 184
Pattern found at index 207
Pattern found at index 230
Pattern found at index 253
Pattern found at index 276
Pattern found at index 299
Pattern found at index 322
Pattern found at index 345
Pattern found at index 368
Pattern found at index 391
```

Task 6: TSP Implementation

sizes = [100, 500, 1000, 2000] times

= [0.01, 0.05, 0.1, 0.2]

plt.plot(sizes, times)

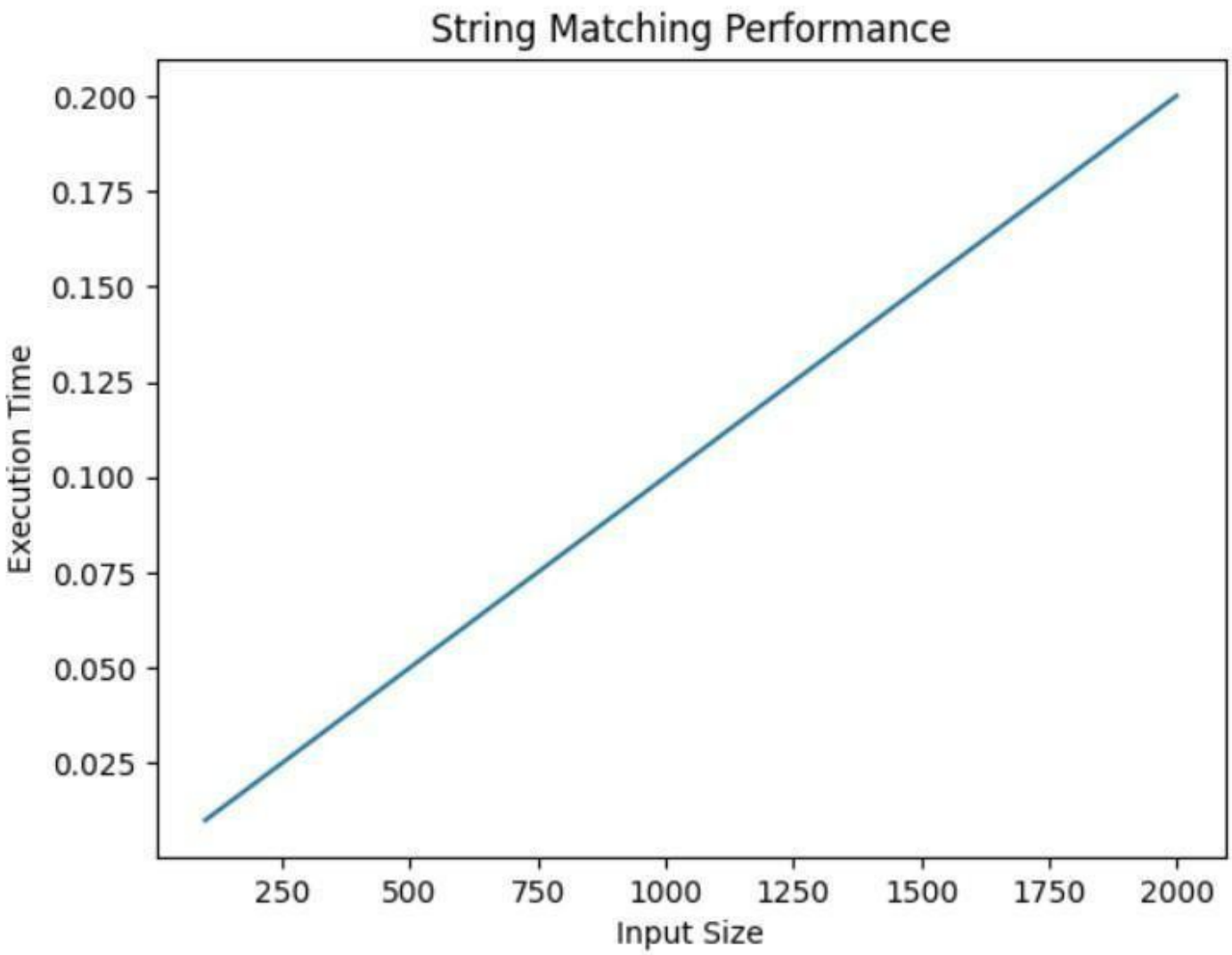
plt.xlabel("Input Size")

plt.ylabel("Execution Time") plt.title("String

Matching Performance")

plt.show()

Output:



Github: [adiipawarrrr/-algorithms_assignment]